

სისტემის ძირითადი პარამეტრები

ის ძირითადი პარამეტრები რის საფუძველზეც მუშაობს სისტემა, ცეცხლის დაფიქსირების და ჩაქრობის საზღვრები, კადრების რაოდენობა ცეცხლის ჩაქრობის დადასტურებისთვის და ტელეგრამზე შეტყობინების გასაგზავნად საჭირო მონაცემები.

```
const float FIRE_ON_THRESHOLD = 80.0f;
const float FIRE_OFF_THRESHOLD = 55.0f;
const int COLD_FRAMES_REQUIRED = 8;
const int COOLING_FRAMES_REQUIRED = 10;
const float COOLING_DELTA_THRESHOLD = -0.2f;
const unsigned long BEEP_INTERVAL_MS = 10000UL;
const unsigned long BEEP_DURATION_MS = 500UL;
const char* TELEGRAM_BOT_TOKEN =
"7655520460:AAG239a4LqNq0acolNI3Pn9RyacG_6AhRUc";
String TELEGRAM_CHAT_ID = "8154736889";
```

სისტემის მდგომარეობები(state-ები) და სხვადასხვა ნაკადებზე უსაფრთხო დადასხვლა

ეს არის პროგრამის ნაკადების უსაფრთხოდ მართვის სისტემა სადაც წარმოდგენილია სისტემის მდგომარეობები. უმოქმედობა, მონიტორინგი, ცეცხლის დაფიქსირება, შეტყობინების გაგზავნა. `currentState` ინახავს სისტემის მიმდინარე მდგომარეობას და `getState()` მას კითხულობს ხოლო `setState()` ფუნქციის მეშვეობით კი უსაფრთხოდ გადადის ერთი მდგომარეობიდან მეორეში და `stateMux` კი ზღუდავს ერთდროულად სისტემის მდგომარეობებზე რამოდენიმე დავალებას(task-ს) შესრულებაში პროცესორს რათა არ წარმოიქმნას ხარვეზები მიმდინარეობის პროცესში `portENTER_CRITICAL()` და `portEXIT_CRITICAL()` ფუნქციების საშუალებით.

```
enum SystemState { STATE_IDLE, STATE_MONITORING, STATE_FIRE_ACTIVE,
STATE_ALARM };
```

```
static SystemState currentState = STATE_IDLE;
static portMUX_TYPE stateMux = portMUX_INITIALIZER_UNLOCKED;
```

```
inline SystemState getState() {
    SystemState s;
    portENTER_CRITICAL(&stateMux);
    s = currentState;
    portEXIT_CRITICAL(&stateMux);
    return s;
}
```

თერმული კამერის მშაობის მექანიზმი

სენსორის ფუნქციონირების მექანიზმი გაშვებულია პროცესორის მეორე ბირთვზე და კითხულობს 768 პიქსელების ტემპერატურების მატრიცას და მაქსიმალური ტემპერატურის მნიშვნელობას გადასცემს სისტემის ძირითად მაკონტროლებელ მექანიზმს.

```
inline void setState(SystemState s) {
    portENTER_CRITICAL(&stateMux);
    currentState = s;
    portEXIT_CRITICAL(&stateMux);
}

void sensorTask(void* parameter) {
    while (true) {
        if (getState() == STATE_IDLE) {
            vTaskDelay(50 / portTICK_PERIOD_MS);
            continue;
        }

        if (mlx.getFrame(sensorFrame) != 0) {
            Serial.println("[SENSOR] Frame read failed – retrying.");
            vTaskDelay(20 / portTICK_PERIOD_MS);
            continue;
        }

        float maxT = sensorFrame[0];
        for (int i = 1; i < 32 * 24; i++)
            if (sensorFrame[i] > maxT) maxT = sensorFrame[i];

        xQueueReset(tempQueue);
        xQueueSend(tempQueue, &maxT, 0);

        vTaskDelay(10 / portTICK_PERIOD_MS);
    }
}
```

ცეცხლის ჩაქრობის ტედექტირება და ტემპერატურის ვარდნის დაფიქსირება

სისტემა ცეცხლის ჩაქრობას აფიქსირებს ორი პარალელური მეთოდის გამოყენებით, პირველი თუ ტემპერატურა ჩამოცდება 55 გრადუს და მეორე ტემპერატურის შენარჩუნების შემთხვევაში აკვირდება გაგრილების თანმიმდევრული ტენდენციების ანალიზს თუ კადრებს შორის ტემპერატურის ვარდნა არის 0,3 გრადუსი ან ნაკლები.

```
case STATE_FIRE_ACTIVE: {
    if (maxTemp > peakTemp) peakTemp = maxTemp;

    float delta = maxTemp - previousTemp;

    if (delta <= COOLING_DELTA_THRESHOLD) {
        coolingFrames++;
    } else {
        coolingFrames = 0;
    }

    previousTemp = maxTemp;

    if (maxTemp < FIRE_OFF_THRESHOLD) {
        coldFrameCount++;
        if (coldFrameCount >= COLD_FRAMES_REQUIRED) {
            setState(STATE_ALARM);
            lastAlertTime = millis();
            startBuzzer();
            triggerAlert(maxTemp);
        }
    } else {
        if (coldFrameCount > 0) {
            coldFrameCount = 0;
        }
    }

    if (coolingFrames >= COOLING_FRAMES_REQUIRED) {
        setState(STATE_ALARM);
        lastAlertTime = millis();
        startBuzzer();
        triggerAlert(maxTemp);
    }

    break;
```

```
}
```

ტელეგრამზე შეტობნების გაგზავნა

შეტყობინების გაგზავნის ამოცანა გაშვებულია პროცესორის პირველ ბირთვზე და დამოუკიდებლად ამუშავებს Telegram API-ს მოთხოვნებს და ამ პროცესში თერმული კამერიდან გამოგზავნილი ინფორმაციის დამუშავების პროცესს არ აფერხებს.

```
void telegramTask(void* parameter) {
    AlertData alert;
    while (true) {
        if (xQueueReceive(alertQueue, &alert, 0) == pdTRUE) {
            if (WiFi.status() == WL_CONNECTED) {
                WiFiClientSecure localClient;
                localClient.setInsecure();
                UniversalTelegramBot bot(TELEGRAM_BOT_TOKEN, localClient);

                String message = "🔥 *Gas Stove Alarm!*\\n";
                message += "The flame has gone out unexpectedly.\\n";
                message += "Last temperature reading: *" + String(alert.temp, 1) + " °C*\\n";
                message += "⚠️ Check the stove immediately — possible gas leak risk!";

                Serial.println("[Telegram] Sending alert...");
                bot.sendMessage(TELEGRAM_CHAT_ID, message, "Markdown");
                Serial.println("[Telegram] Alert sent.");
            } else {
                Serial.println("[Telegram] Skipped – no Wi-Fi.");
            }
        }
        vTaskDelay(50 / portTICK_PERIOD_MS);
    }
}
```

სრული კოდის github-ის ლინკი:

<https://github.com/MateMch-iliauni/Project-Codes/blob/main/finalproject.ino>